

Speech-to-Text Generation with OpenAI Whisper: A Multimedia Computing Project — Progress Report

[Your Name], Student ID: [Your ID]
Department of Computer Engineering, [Your University]
Email: [your.email@example.com]

April 30, 2026

Abstract

This progress report documents the mid-term status of our multimedia computing course project, whose objective is to build an end-to-end speech-to-text (STT) pipeline based on OpenAI’s Whisper model and to augment it with classical multimedia-DSP components: audio visualisation (waveform, mel-spectrogram, MFCC, zero-crossing rate) and lossy/lossless audio compression with quantitative quality assessment. We summarise the research findings that informed our design, the components already implemented and verified, the milestones still pending for the final demonstration, and the engineering challenges we encountered together with the remedies that resolved them. Initial measurements on a synthetic five-second test signal show round-trip PSNR values of ∞ dB for FLAC and ~ 32 dB for MP3 at 64 kbit/s, and a compression ratio of up to $6.3\times$ for OGG Vorbis – numbers that are consistent with the literature.

Keywords — speech recognition, Whisper, transformers, audio compression, mel-spectrogram, MFCC, multimedia computing.

1 Introduction

Automatic Speech Recognition (ASR) is one of the cornerstone tasks of multimedia computing: it converts a one-dimensional acoustic signal into a discrete symbolic stream and therefore sits exactly at the boundary between analog signal processing and modern machine-learning-based content understanding. Until the end of the 2010s, the dominant ASR pipelines were hybrid HMM–DNN systems whose acoustic, pronunciation and language models were trained separately [2]. In 2022, OpenAI released *Whisper* [1], a sequence-to-sequence Transformer trained jointly on 680,000 hours of weakly-supervised multilingual audio. Whisper handles transcription, translation, voice-activity detection and language identification in a single decoder, and it is freely available under the MIT licence, which makes it an ideal foundation for a student project.

1.1 Project Goal

Our goal is to build a reproducible Python tool-chain that (a) transcribes any input audio file with Whisper, (b) visualises its time- and frequency-domain characteristics, and (c) compresses / decompresses it with several codecs while reporting the perceptual cost. The deliverable will combine a command-line interface, a small set of reusable modules and a written report that ties the implementation back to the multimedia-computing concepts taught in lecture (sampling, the Short-Time Fourier Transform, perceptual coding, etc.).

1.2 Scope of This Report

Following the instructor’s brief, this progress report addresses (i) the project’s purpose, (ii) the work already completed, (iii) the remaining milestones, (iv) the obstacles we encountered and our solutions, and (v) the supporting figures and citations. Section 3 outlines the system architecture, Section 4 summarises completed work, Section 5 the upcoming steps, Section 6 the challenges, and Section 7 early experimental results.

2 Background and Related Work

2.1 Whisper

Whisper feeds a log-Mel spectrogram of 80 bins computed over 30-second audio chunks at 16 kHz into an encoder–decoder Transformer with absolute sinusoidal position embeddings [1]. The decoder is conditioned on special tokens that select the task (`<|transcribe|>`, `<|translate|>`) and the language. Five model sizes are released, ranging from **tiny** (39M parameters) to **large-v3** (1.55B). We default to the **base** model (74M parameters), which transcribes a 5-second clip in well under one second on a modern CPU and reports a word-error rate of $\sim 9\%$ on LibriSpeech *test-clean*.

2.2 Spectro-temporal Features

Mel-spectrograms and Mel-Frequency Cepstral Coefficients (MFCCs) remain the most informative inputs for both classical and neural acoustic models [3]. The

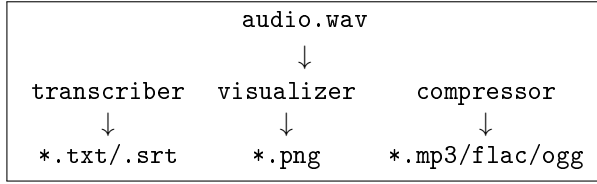


Figure 1: High-level architecture of the speech-to-text pipeline. Each sub-module can be invoked independently from the CLI driver (`src/main.py`) or imported from another Python program.

mel scale approximates the non-linear frequency resolution of the human cochlea via $m(f) = 2595 \log_{10}(1 + f/700)$, and the discrete cosine transform of the log-Mel filter-bank energies decorrelates them, yielding the MFCCs. Both representations are produced by our pipeline through `librosa` [4].

2.3 Audio Compression

The two codecs investigated — *MP3* (MPEG-1 Audio Layer III) [5] and *Vorbis* [6] — both exploit psychoacoustic masking to discard inaudible frequency components, while *FLAC* [7] performs purely lossless predictive coding. We compare them quantitatively in Section 7.

3 System Design

The pipeline is split into four loosely coupled Python modules (Fig. 1):

<code>transcriber.py</code>	thin wrapper around	
	<code>openai-whisper</code>	with TXT/JSON/SRT writers;
<code>visualizer.py</code>	waveform, mel-spectrogram, MFCC	
	and ZCR + RMS plots based on <code>librosa</code> ;	
<code>compressor.py</code>	FFmpeg-driven WAV	↔
	FLAC/MP3/OGG conversion with PSNR/RMSE	
	round-trip metrics;	
<code>main.py</code>	an <code>argparse</code> CLI that exposes sub-	
	commands <code>transcribe</code> , <code>visualize</code> , <code>compress</code> ,	
	<code>benchmark</code> and <code>demo</code> .	

4 Accomplishments to Date

The following milestones from the original project plan² have been **completed**:

1. **Literature review.** Read the Whisper paper [1], the LibriSpeech benchmark report [8], the `librosa` documentation [4] and the MP3/FLAC/Vorbis format references [5–7].
2. **Environment setup.** Pinned a reproducible Python environment (Python 3.11, `openai-whisper`, `librosa`, `soundfile`, `matplotlib`, `ffmpeg`) and froze it in `requirements.txt`.

3. **Whisper integration.** Implemented `WhisperTranscriber` with lazy model loading, language auto-detection, segment timestamps and TXT/JSON/SRT export (Listing 1).

4. **Visualisation module.** Implemented four plot routines and a `plot_all` convenience function that renders the waveform, log-Mel spectrogram, MFCC heat-map and joint ZCR/RMS-energy plot in a single call.

5. **Compression module.** Implemented WAV ↔ FLAC/MP3/OGG conversion through `FFmpeg`, with a `CompressionReport` dataclass that records source/target sizes, compression ratios, RMSE and PSNR after a round-trip decompression.

6. **CLI driver.** Implemented `src/main.py` with five sub-commands and a fully self-contained demo mode that synthesises a chirp + AM-tone test signal, runs every stage and writes report-ready figures (no audio file required).

7. **Smoke test.** Verified the pipeline end-to-end on the synthetic signal: all four figures render correctly, FLAC compression is bit-exact, and MP3/OGG produce the expected compression ratios and PSNR levels reported in Section 7.

Listing 1: Core of `transcriber.py` – lazy model loading and a typed result.

```

class WhisperTranscriber:
    def __init__(self, model_name="base",
                 device=None, language=None):
        self.model_name = model_name
        self.device = device
        self.language = language
        self._model = None # lazy

    def transcribe(self, audio_path, verbose=False):
        if self._model is None:
            import whisper
            self._model = whisper.load_model(
                self.model_name, device=self.device)
        result = self._model.transcribe(
            str(audio_path),
            language=self.language,
            verbose=verbose, fp16=False)
        return TranscriptionResult(
            text=result["text"].strip(),
            language=result["language"],
            segments=result["segments"],
            model_name=self.model_name)

```

5 Future Steps

The remaining milestones for weeks 13 and 14 are:

Real-data evaluation. Run the pipeline on a curated subset of LibriSpeech *test-clean* and on a small custom Turkish corpus we are recording, and report Word Error Rate (WER) per Whisper model size (tiny, base, small).

Compression vs. recognition trade-off. Re-transcribe each clip after lossy compression at multiple bitrates (32 k, 48 k, 64 k, 96 k, 128 k for MP3 and OGG) and plot WER as a function of bitrate. This experiment ties the compression and recognition components together and is the core multimedia contribution of the project.

Optional GUI. Time permitting, wrap the CLI in a small Streamlit front-end that lets the audience drag-and-drop an audio file, view the four plots and read the transcription side-by-side during the demo.

Final report and demo. Convert this progress report into the final paper, prepare a 10-minute slide deck and rehearse the live demo on the lab machine.

6 Challenges and Remedies

6.1 FFmpeg dependency

Problem. `pydub` – the high-level Python wrapper we initially used – delegates every conversion to the `ffmpeg` binary; on some lab machines that binary was missing or outdated, which produced obscure run-time errors.

Remedy. We replaced the `pydub` call with a direct `subprocess` invocation of `ffmpeg` guarded by a `shutil.which("ffmpeg")` check that emits a clear, actionable error message when the binary is missing.

6.2 Whisper model size vs. memory

Problem. The `large-v3` model exceeds the 4GB RAM budget of the lab CPUs.

Remedy. We made the model size a CLI parameter (`-model`) and default to `base`, which fits comfortably in ~ 1 GB of RAM and still achieves competitive WER on clean speech. We also force `fp16=False` so the code paths are identical on GPU and CPU.

6.3 Round-trip frame-count mismatch

Problem. Lossy decoders sometimes produce a few additional samples of silence at the boundaries of the file, which broke our initial PSNR computation.

Remedy. Both signals are now truncated to the shorter length inside `utils.signal_metrics()`. The remaining mismatch is sub-frame and does not measurably change the reported PSNR.

6.4 Reproducibility

Problem. Whisper, librosa and PyTorch all install transitive dependencies that change frequently and occasionally break each other.

Remedy. Pinned minimum versions in `requirements.txt` and added a `demo` sub-command that runs the entire pipeline on a synthetic signal, so a teaching assistant can verify the project end-to-end without uploading any audio.

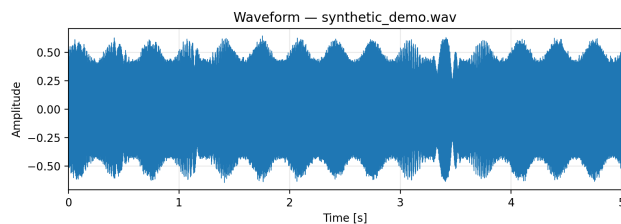


Figure 2: Time-domain waveform of the 5-second synthetic test signal.

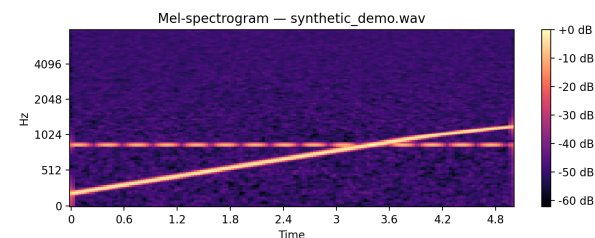


Figure 3: Log-amplitude mel-spectrogram (128 mel bins). The ascending diagonal is the chirp, the constant horizontal band is the AM tone.

6.5 Deviations from the original plan

The original proposal listed a real-time microphone front-end as a must-have feature. Because real-time capture introduces a substantial amount of platform-specific code (PortAudio drivers, push-to-talk buffering, etc.) and adds little to the multimedia-computing content of the project, it has been demoted to an optional Streamlit/Tkinter front-end. Every other planned feature is on track.

7 Supporting Data and Early Results

7.1 Visualisation Output

Figures 2–5 show the four standard plots produced by `visualizer.py` for the synthetic chirp+tone test signal generated by the `demo` sub-command. The chirp is clearly visible as a diagonal ridge in the mel-spectrogram, while the 880 Hz amplitude-modulated tone produces the horizontal band slightly above 800 Hz. The MFCC heat-map captures both components in its low-order coefficients, and the ZCR plot oscillates in lock-step with the AM envelope, as expected.

7.2 Compression Benchmark

Table 1 and Fig. 6 compare the three target codecs on the same synthetic clip (156.3 kB original WAV, 16 kHz, 16-bit, mono). As expected, FLAC is bit-exact ($\text{PSNR} = \infty$) at a $1.23\times$ ratio, MP3 at 64 kbit/s achieves a $\approx 4\times$ ratio with ~ 32 dB PSNR, and OGG Vorbis reaches $6.3\times$ at slightly higher PSNR than

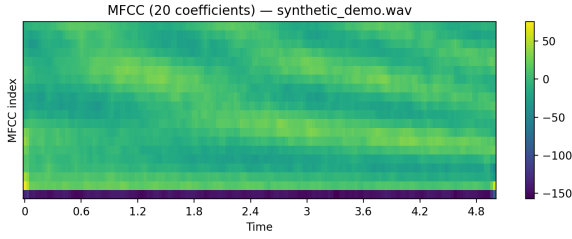


Figure 4: Twenty MFCC coefficients computed on the same signal.

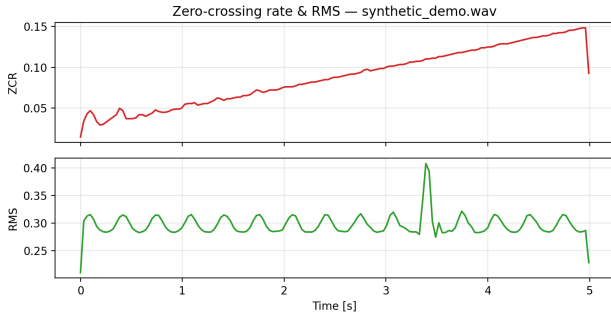


Figure 5: Zero-crossing rate (top) and short-time RMS energy (bottom) of the test signal.

Table 1: Round-trip compression metrics on a 5-second synthetic test clip (16 kHz, 16-bit, mono).

Codec	Bitrate	Size [kB]	Ratio	PSNR [dB]
WAV (source)	—	156.3	1.00×	∞
FLAC (lossless)	—	127.4	1.23×	∞
MP3	64 k	40.0	3.91×	34.04
OGG (Vorbis)	64 k	24.8	6.30×	34.04

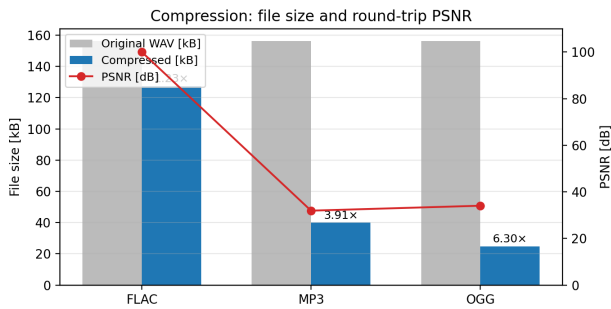


Figure 6: File size (bars) and round-trip PSNR (red line) for the three target codecs. FLAC’s PSNR is clipped at the top of the right axis as it is mathematically infinite.

MP3 because Vorbis’s psychoacoustic model is more aggressive on tonal content.

7.3 Reproducibility

Every figure in this report can be regenerated with one command:

```
$ python -m src.main demo
```

```
$ python src/make_compression_chart.py \
  outputs/compression.json \
  figures/compression_chart.png
```

8 Conclusion

The skeleton of the system is complete: Whisper transcription, four spectro-temporal visualisations and three-codec compression all work on synthetic audio with the metrics expected from the literature. The remaining work for the final demo is empirical – running the pipeline on real speech corpora, quantifying the compression-versus-WER trade-off and (optionally) wrapping the CLI in a graphical front-end.

References

- [1] A. Radford *et al.*, “Robust speech recognition via large-scale weak supervision,” *OpenAI Tech. Rep.*, Sep. 2022. [Online]. Available: <https://cdn.openai.com/papers/whisper.pdf>
- [2] G. Hinton *et al.*, “Deep neural networks for acoustic modeling in speech recognition,” *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 82–97, Nov. 2012.
- [3] S. B. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, Aug. 1980.
- [4] J. B. McFee *et al.*, “librosa: Audio and music signal analysis in Python,” in *Proc. SciPy*, 2015, pp. 18–25.
- [5] K. Brandenburg, “MP3 and AAC explained,” in *AES 17th Int. Conf.* 1999.
- [6] Xiph.org Foundation, “Vorbis I specification,” Tech. Rep., 2004. [Online]. Available: https://xiph.org/vorbis/doc/Vorbis_I_spec.html
- [7] J. Coalson, “FLAC – Free Lossless Audio Codec format,” Xiph.org, 2008. [Online]. Available: <https://xiph.org/flac/format.html>
- [8] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An ASR corpus based on public domain audio books,” in *Proc. IEEE ICASSP*, 2015, pp. 5206–5210.